Autonomous Systems - CSL7650

Assignment - 1

Negotiation-Based Plan Repair for Multi-Agent Warehouse Navigation

Source Code: https://github.com/Sahil-1918912/PathWorks

Sarah Fatima (B23EE1093) Sahil Narkhede (B23CS1060)
Anshit Agarwal (B23CS1087) Namya Dhingra (B23CS1040)

October 4, 2026

Abstract

Autonomous warehouse systems rely on coordinated multi-agent pathfinding to move goods efficiently through shared floor space. In practice, however, the environment changes during execution: robots may break down, cells may become blocked, or urgent tasks may appear. Re-running a full multi-agent planner after every disruption is computationally expensive and can unnecessarily alter the routes of many agents. This project addresses this challenge through a negotiation-based plan repair framework for dynamic warehouse navigation. A team of robots initially plans paths with space-time A*; when a disruption occurs, only the affected agents repair their plans, negotiating with neighbors when local repair is not sufficient. The approach preserves the original joint plan wherever possible, while minimizing the number of agents whose routes are changed. The system is evaluated using total completion time and repair impact, and the results show that local repair achieves near-baseline performance while substantially reducing plan changes compared with full replanning.

Contents

1 Introduction **3**

2 Project Overview **3**

3 System Model **4**

3.1 Warehouse Grid 4

3.2 Tasks and Agents 4

3.3 Disruption Types 4

4 Initial Planning **4**

5 Algorithm **4**

6 Plan Repair Strategy **5**

7 Experimental Methodology **6**

8 Evaluation Results **6**

8.1 Team Size Experiments 6

8.2 Dynamic Obstacle Density Experiments 8

8.3 Disruption-Type Breakdown 9

9 Novelty: Heterogeneous Robot Speeds in the Warehouse **10**

10 Discussion **11**

11 Conclusion **12**

12 References **12**

1 Introduction

Multi-agent pathfinding (MAPF) is widely used in warehouse robotics to coordinate the motion of many mobile robots across shared corridors and workstations. In static environments, algorithms such as space-time A* can produce collision-free plans efficiently. However, real warehouses are dynamic: robot failures, temporary cell blockages, and emergency tasks can invalidate the original plan while execution is underway.

If the system re-plans the entire team from scratch every time a disruption occurs, the computational cost can be high and the resulting routes may disturb many robots that were not directly involved in the problem. For practical warehouse systems, a better solution is local repair: react to the disruption, modify only the affected part of the plan, and preserve the existing coordination of unaffected robots wherever possible.

This project implements a negotiation-based plan repair framework for a warehouse grid. Each robot plans its route through a 2D grid map while carrying out pickup and delivery tasks. During execution, random disruptions are introduced, and the system repairs the plan by escalating from self-repair to neighbor negotiation and, if needed, broader replanning. The system is evaluated by total completion time and by the number of agents whose plans must be altered by a single disruption. The project also studies how performance changes as the number of agents and the density of dynamic obstacles increase.

2 Project Overview

The project is organized around a small set of modules:

- **main.py**: command-line entry point for demonstration and experiments.
- **code/grid.py**: warehouse map generation, static layout, dynamic blocks, and distance maps.
- **code/planner.py**: space-time A* planner and reservation table used for path generation.
- **code/agent.py**: robot and task definitions, including goals and execution state.
- **code/simulation.py**: initial planning, execution loop, disruption handling, and summary metrics.
- **code/repair.py**: plan repair logic, local negotiation, and auction-based reassignment.
- **code/scenario.py**: scenario generation and random event creation.
- **code/visualize.py**: animation of the warehouse run.
- **code/experiments.py**: parameter sweeps and CSV/plot generation.

The warehouse map is a 32×20 grid with 4-connected movement, including wait actions. Each robot has a home station and a set of tasks composed of pickup and delivery operations. A robot may not occupy the same cell as another robot at the same time, and edge conflicts are also prevented. The system models a realistic warehouse environment where tasks are allocated to agents and paths are planned in a time-expanded state space.

3 System Model

3.1 Warehouse Grid

The map contains a structured corridor layout with aisle cells and a surrounding route network. Robots move one cell per time step or may wait if needed. Obstacles can be static or dynamic; dynamic obstacles model the kinds of temporary blockages that commonly arise in busy warehouses.

3.2 Tasks and Agents

Each agent is assigned a sequence of tasks, where each task is defined as a pickup cell and a delivery station. After finishing all tasks, a robot may return to its home position. The planner uses the ordered remaining goals to generate a path that respects the current reservations of other agents.

3.3 Disruption Types

The simulator includes three major disruption classes:

- **Blocked cell**: a warehouse cell becomes unusable temporarily or permanently.
- **Breakdown**: a robot stops moving, and its unfinished tasks are reassigned if needed.
- **Emergency task**: a robot is required to perform a high-priority pickup and delivery before continuing its previous work.

These disruptions are deliberately chosen to reflect realistic operational events in automated warehouses.

4 Initial Planning

The initial plan is generated with a prioritized multi-agent planner based on space-time A*. The planner treats earlier agents as fixed reservations, then computes a path for each remaining robot while avoiding collisions with the already scheduled agents. This produces a valid initial joint plan from the start positions to the assigned task goals.

The core planning routine is implemented in `code/planner.py`. Each agent plans through its remaining goals while respecting a reservation table of occupied cells in time. This allows the algorithm to generate safe and efficient paths before disruptions are introduced.

5 Algorithm

The assignment requires an algorithm that matches a dynamic warehouse scenario in which a team of robots moves on a 2D grid and must complete pickup-and-delivery tasks while responding to unexpected events. The project addresses this requirement with a two-stage process: initial multi-agent pathfinding followed by local repair when execution is disrupted.

In the first stage, every robot computes an initial route using space-time A* over the warehouse grid. The route includes the sequence of pickup locations, delivery stations, and return-to-home movement. The planner records reservations for occupied cells over time, so the resulting plan is collision-free for the nominal operating environment. This stage corresponds directly to the assignment requirement that agents work out their paths using a multi-agent pathfinding algorithm.

In the second stage, the system handles dynamic contingencies that occur during execution. These events include robot breakdowns, blocked cells, and emergency tasks. When such an event happens, the affected robot does not re-run the full multi-agent planner from scratch. Instead, it identifies the portion of its remaining route that is invalid and attempts a local repair. The robot first replans its own remaining path while treating all unaffected robot plans as hard reservations. If the self-repair route is too slow or impossible, the robot asks nearby robots to negotiate alternative paths. The negotiation is local: only agents in the disrupted region or within communication range are considered. This matches the assignment requirement that disrupted agents communicate with neighbouring agents to negotiate mutually acceptable alternative paths.

The repair process is intentionally incremental. It escalates only as much as necessary: the robot tries to fix the issue alone, then with nearby agents, then with a broader set if required. A repair is accepted only if it is sufficiently beneficial compared with the original local plan, and the system aims to preserve the original routes of unaffected robots. The objective is therefore not merely to find any feasible plan, but to minimize the number of agents whose original plans are altered. This is the central requirement in the assignment brief.

Formally, the system minimizes the impact metric

$$\text{RepairImpact} = \sum_{i=1}^{n} \mathbf{1}(P_i^{\text{new}} \neq P_i^{\text{old}}), \tag{1}$$

where P_i^{old} is the original path of agent i and P_i^{new} is the path after local repair. This criterion is applied while also optimizing total completion time, so the algorithm balances efficiency with minimal disruption.

The assignment also requires the system to be evaluated with respect to time and repair impact. Accordingly, the project measures total time steps for all agents to complete their assigned work and reports the mean number of agents whose original plans must be modified after a single disruption. It also varies both the number of agents and the density of dynamic obstacles to observe how the system behaves under increasing congestion and uncertainty.

6 Plan Repair Strategy

The repair framework is implemented in `code/repair.py`. The key principle is that the system should not re-run the global multi-agent planner whenever a local disruption occurs. Instead, the affected agent attempts repair in escalation levels:

1. **Self-repair:** the affected robot replans its remaining route while treating other robots' current paths as hard reservations.

2. **Neighbor negotiation:** if the self-repair route is too costly or infeasible, the robot negotiates with nearby agents whose planned paths conflict with the proposed detour.

3. **Broader repair:** the robot attempts a wider negotiation with more agents if necessary.

4. **Safe stop:** if no feasible repair is found, the robot holds position and retries with backoff.

When a robot breaks down, unfinished tasks are reassigned via a contract-net-style auction, favoring robots that have already changed their plan or are nearby, which preserves coordination and reduces unnecessary changes.

A crucial design objective is the minimization of repair impact:

$$\text{RepairImpact} = \sum_{i=1}^{n} \mathbf{1}(P_i^{\text{new}} \neq P_i^{\text{old}}). \tag{2}$$

This metric counts the number of agents whose original plans are altered by the disruption. The project explicitly compares this against the baseline full replanning approach.

7 Experimental Methodology

The project runs controlled experiments over multiple random seeds and parameter settings. The evaluation sweep includes:

- varying number of agents,
- varying dynamic obstacle density,
- comparison of three methods: negotiated repair, full replanning, and self-repair-only.

The project executes 780 simulations in total, as described in the repository documentation. Each run records disruptions, affected agents, changed plans, negotiation messages, and total completion time. The key metrics are:

- **Total time steps**: the sum of the completion times of all agents.
- **Plans changed per disruption**: the number of agents whose routes are modified after repair.
- **Overhead percentage**: increase relative to the initial no-disruption plan.
- **Emergency response time**: time to complete urgent tasks.

8 Evaluation Results

8.1 Team Size Experiments

The project evaluates the effect of team size at fixed obstacle density of 5%. Figure 1 and Figure 2 summarize the trend.

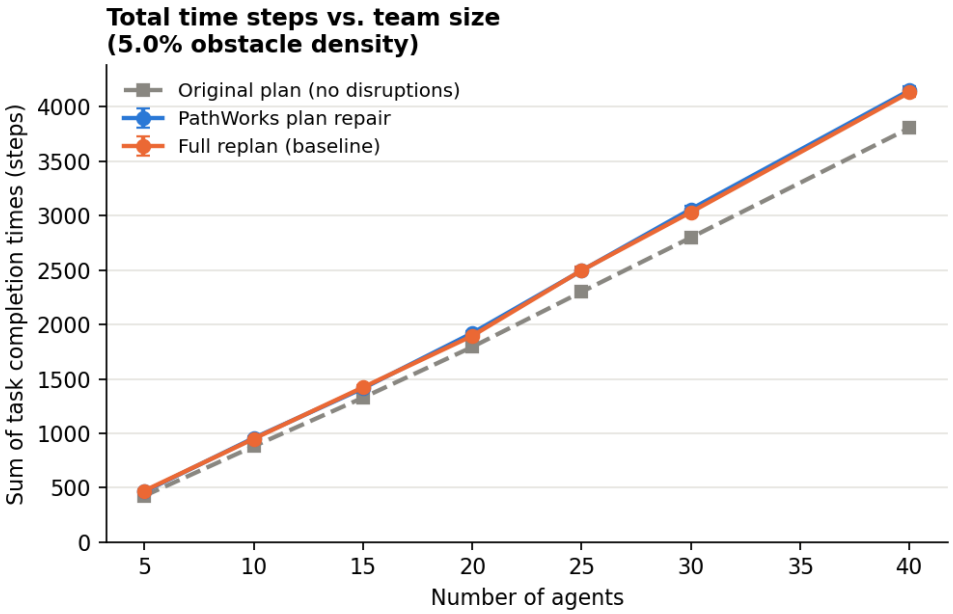


Figure 1: Total time steps versus number of agents at 5% obstacle density.

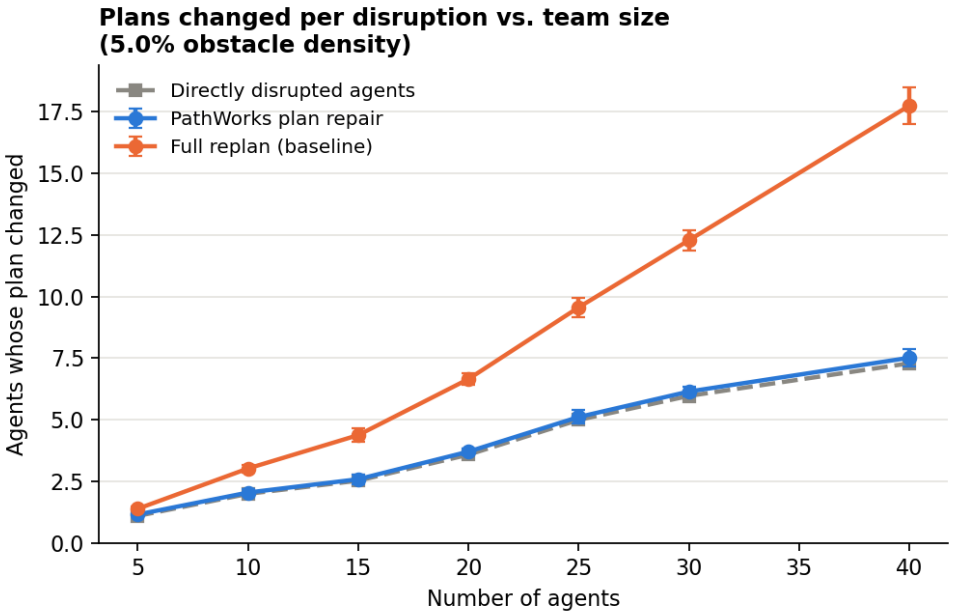


Figure 2: Plans changed per disruption versus number of agents at 5% obstacle density.

The experimental dataset in `results/summary.csv` reports the following averages for team-size variation:

Table 1: Impact of team size on performance at 5% obstacle density.

Robots	Original Plan	Repair Total	Replan Total	Repair Changed	Replan Changed
5	425.0	464.7	471.9	1.16	1.39
10	880.6	956.3	950.0	2.04	3.02
15	1327.7	1413.4	1422.6	2.59	4.37
20	1794.1	1921.4	1896.3	3.70	6.65
25	2300.4	2495.5	2497.8	5.11	9.56
30	2799.6	3060.2	3033.4	6.14	12.28
40	3807.1	4152.6	4134.2	7.51	17.74

These values show a consistent pattern: the repair method keeps total completion time close to the full replanning baseline, while changing far fewer original plans. The gap in repair impact becomes more pronounced as the number of robots grows.

8.2 Dynamic Obstacle Density Experiments

The project also evaluates performance as the density of dynamic obstacles increases for a fixed team of 20 robots. Figure 3 and Figure 4 show the corresponding trends.

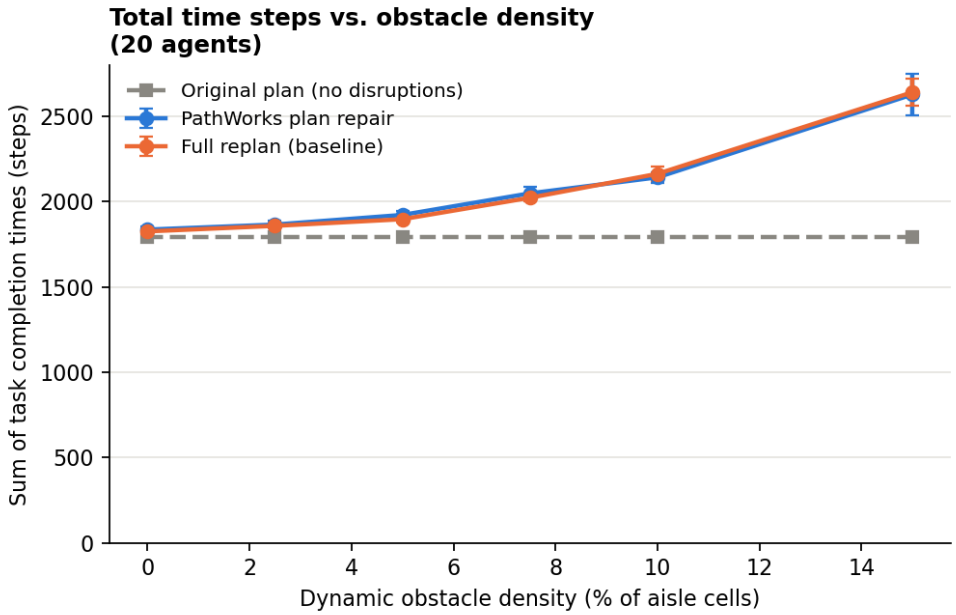


Figure 3: Total time steps versus obstacle density with 20 agents.

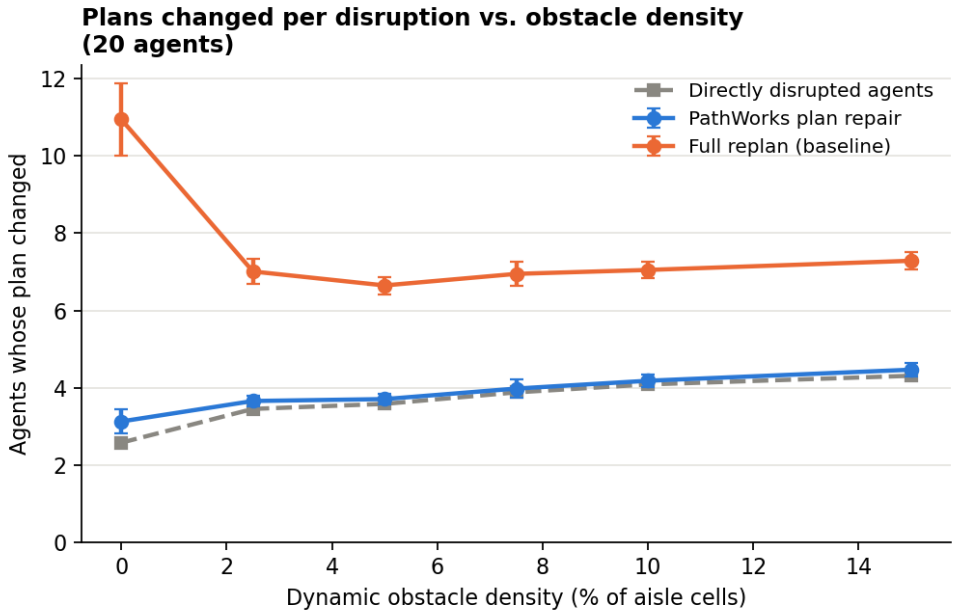


Figure 4: Plans changed per disruption versus obstacle density with 20 agents.

The tabulated results are as follows:

Table 2: Impact of dynamic obstacle density on performance with 20 agents.

Density	Repair Total	Overhead	Replan Total	Repair Changed	Replan Changed
0%	1835.4	2.3%	1824.9	3.12	10.95
2.5%	1864.9	3.9%	1857.3	3.65	7.01
5%	1921.4	7.1%	1896.3	3.70	6.65
7.5%	2047.9	14.2%	2022.1	3.98	6.95
10%	2141.5	19.5%	2162.1	4.18	7.05
15%	2627.1	46.6%	2640.1	4.47	7.29

This shows that higher obstacle density increases the overhead dramatically. The repair method remains competitive with full replanning in total time, while the number of altered plans remains modest and nearly constant compared with the much larger number induced by full replanning.

8.3 Disruption-Type Breakdown

The project also records plan changes by disruption type. The generated summary plot is shown in Figure 5.

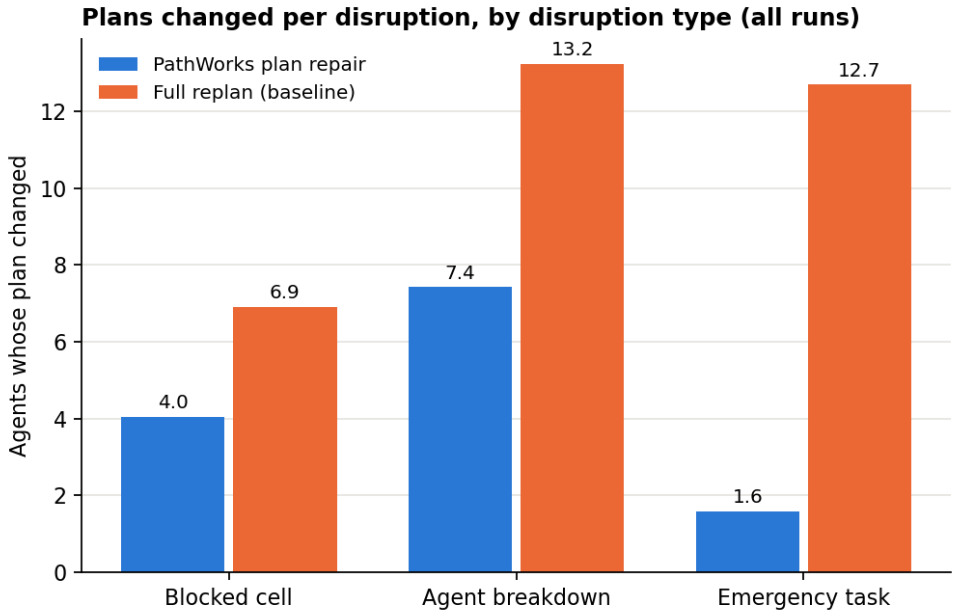


Figure 5: Average plans changed per disruption by disruption type.

The repository results indicate that repair changes approximately 4.1 plans for blocked-cell events, 7.4 plans for breakdowns, and 1.6 plans for emergency tasks. Full replanning changes much larger numbers, often roughly double or more, showing that repair is especially beneficial when a local disruption occurs in a congested environment.

9 Novelty: Heterogeneous Robot Speeds in the Warehouse

A key contribution of this project is that the warehouse is not modeled as a homogeneous fleet in which all robots move at the same speed. Instead, each robot is assigned its own execution pace, so agents can move at different velocities while still following the same planned route structure. This makes the task more realistic than standard MAPF formulations, where all robots are usually assumed to advance one grid cell per time step.

In the implementation, each robot has a speed parameter p_i sampled uniformly from a bounded range:

$$p_i \sim \mathcal{U}(p_{\min}, p_{\max}),$$

with typical values in the project set to $p_{\min} = 0.8$ and $p_{\max} = 1.8$. The path itself remains a sequence of grid cells, but the actual execution time of the path is scaled by the robot's pace. Formally, the cell occupied by robot i at simulation time t is obtained by

$$x_i(t) = \pi_i(\lfloor t \cdot p_i \rfloor),$$

where π_i is the path of robot i. This means that a faster robot can traverse the same route in fewer simulation steps, while a slower robot consumes more time on the same path.

This design introduces a more realistic and challenging multi-agent setting because the system must now handle asynchronous robot motion in addition to collision avoidance and

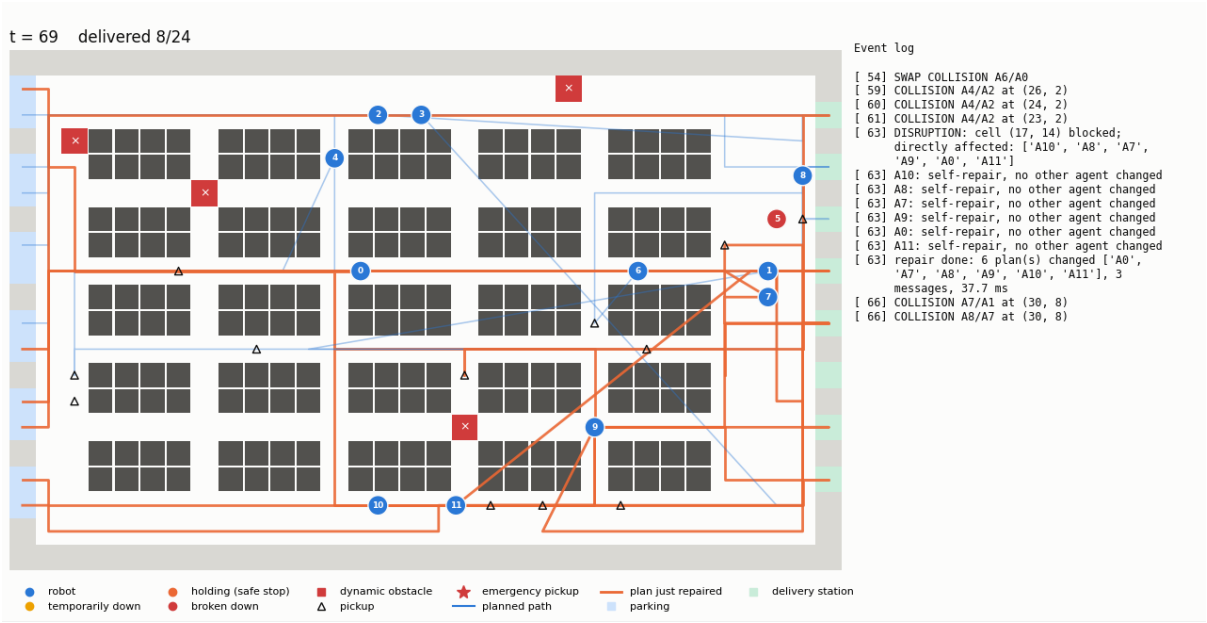


Figure 6: Image of the simulation created.

dynamic disruption repair. The planner still uses a time-expanded reservation table, but the timing of future occupancy is now agent-dependent rather than uniform. As a result, the repair process must consider heterogeneous delays while re-routing robots after blockages, breakdowns, or emergency task insertion.

The novelty is therefore twofold:

- the classical MAPF setting is extended to a heterogeneous robot team;
- the repair framework remains local and incremental, without re-running the full multi-agent plan generation from scratch.

This is particularly important in dynamic warehouse environments, where some robots may be faster and able to clear bottlenecks sooner, while others are slower and may become more likely sources of congestion. The project therefore models a more realistic operational scenario than a fully synchronized multi-robot path planner and evaluates how local negotiation-based repair behaves under such heterogeneous motion.

10 Discussion

The results show that local repair is an effective strategy for dynamic multi-agent warehouse navigation. In almost all cases, the repair method yields a completion time close to the full replanning baseline. This result is particularly important because it shows that local repair is not merely a workaround; it can preserve solution quality while avoiding widespread re-optimization.

The more significant advantage is in the number of agents whose plans change. Repair modifies only the minimally necessary set of routes, whereas full replanning often changes many agents, even when they were not directly affected by the disruption. The increased disruption

caused by full replanning becomes more severe as the number of agents rises, confirming the need for local repair in practical multi-agent warehouses.

The project also observes that obstacle density strongly affects total time. At low obstacle rates, the total completion time stays relatively close to the undisturbed plan. At 15% dynamic density, the overhead becomes large, which reflects the increased need for detours and route restructuring. However, even in this difficult regime, repair still remains competitive with full replanning while affecting fewer agents.

The negotiation stage is important for difficult cases. The repository notes that about 96% of disrupted robots repair themselves at the first level; the remaining agents require negotiation with neighbors to find feasible alternatives. This is consistent with the observation that local negotiation prevents deadlocks and improves completion reliability.

11 Conclusion

This project demonstrates a practical and effective approach to dynamic multi-agent plan repair in warehouse robotics. By repairing only the affected portion of the plan instead of re-running the entire multi-agent planner, the system preserves most of the original coordination while adapting to unexpected disruptions. The results show that the repair method achieves nearly the same total completion time as full replanning, while affecting far fewer robots and maintaining robust completion under dynamic obstacles and emergencies.

The project therefore supports the idea that plan repair is a highly suitable strategy for autonomous warehouses, where robustness, throughput, and operational continuity are critical. The combination of local repair, neighbor negotiation, and task reassignment forms a scalable and efficient framework for dynamic warehouse operations.

12 References

References

[1] Stern, R., et al. “Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks.” AAMAS, 2019.

[2] Silver, D. “Cooperative Pathfinding.” AIIDE, 2005.

[3] Standley, T. “Finding Optimal Solutions to Cooperative Pathfinding Problems.” AAAI, 2010.

[4] Wagner, F., and R. Choset. “Subdimensional Expansion for Multirobot Path Planning.” AI Journal, 2015.